



CS-2009 Design and Analysis of Algorithms – 2025

ASSIGNMENT-03

Instructions

Programming & Submission Guidelines

1. Separate Source Files for Each Question

- You must submit a **separate C++ source file** for each problem in this assignment.
- Follow the exact naming convention below:
- Q<question_number>_<roll_number>.cpp

Example:

Q1_23I-0456.cpp

Q2_23I-0456.cpp

2. Report Submission

- Prepare **one consolidated report (PDF)** covering **all four questions**.
- Your report must include for each problem:
 - Algorithm explanation and theoretical background
 - Pseudocode of your approach
 - Time and space complexity analysis
 - Diagrams, flowcharts, or transition tables if applicable
 - Output screenshots or sample test cases demonstrating correctness
- The report file name should follow this format:
- Report_23I-XXXX_A3.pdf

3. ZIP File Submission

- Create **one ZIP file** containing:
 - All four .cpp source files
 - The single consolidated PDF report
- Name your ZIP file exactly as:
- 23I-XXXX-A3.zip

Example: 23I-0456-A3.zip

4. Submission Platform

- Submit the ZIP file on **Google Classroom** before the deadline.
- Late submissions will **not be accepted** under any circumstances.

5. General Coding Requirements

- Code must be modular, well-commented, and easy to read.
- Use standard input/output (no hard-coded file paths).
- Each program should be tested against multiple input cases.
- Handle edge cases and invalid inputs gracefully.
- Ensure time complexity aligns with algorithmic expectations for each problem.

6. Plagiarism Policy

- All submissions will be checked for plagiarism.
- Identical or near-identical submissions will result in **zero marks** for all parties involved.

7. Deadline

- **Submission Deadline: 13-Nov-2025**
 - No extensions will be granted.
-



Problem - 1: Adaptive String-Matching Strategy

You are designing a hybrid string matching system that can automatically select between **Naïve**, **KMP**, and **Rabin–Karp** algorithms depending on the characteristics of the input text and pattern.

Task:

Propose an algorithmic framework that decides:

1. When to use KMP over Rabin–Karp or Naïve based on pattern length, alphabet size, and expected collision rate.
2. How to minimize preprocessing time when multiple patterns are searched simultaneously.
3. How to adaptively switch methods during runtime if the text exhibits periodic patterns.

Deliverables:

- Detailed decision criteria (in tabular or rule-based form).
 - Time complexity analysis for each scenario.
 - Example scenario demonstrating your algorithm's behavior.
 - Code Implementation
-

Problem - 2: Divide and Conquer Text Similarity Estimation

Two large documents **A** and **B** (each with millions of words) need to be compared for plagiarism. You are to design a **divide and conquer algorithm** to compute the **maximum common substring length** between the two documents without comparing every substring pair.

Task:

1. Propose an algorithm that divides both texts efficiently.
2. Show how partial results from subproblems can be merged.
3. Analyze its time and space complexity.
4. Discuss how this method could be parallelized across processors.

Deliverables:

- Time complexity analysis for each scenario.
 - Example scenario demonstrating your algorithm's behavior.
 - Code Implementation
-



Problem - 3: Optimal Multi-pattern Matching Automaton

You are tasked with building a **multi-pattern string matcher** that supports **wildcards (?)**, **case-insensitivity**, and **pattern overlap** (like detecting “ana” in “banana”).

However, building the full automaton for all patterns would exceed available memory.

Task:

1. Propose a **memory-optimized design** for the automaton (conceptually).
2. Suggest how **divide and conquer construction** could reduce preprocessing time.
3. Implement the Aho–Corasick
4. Compare its efficiency with Aho–Corasick in both space and time.
5. Provide pseudocode for building and merging partial automata.

Deliverables:

- Time complexity analysis for each scenario.
- Example scenario demonstrating your algorithm’s behavior.
- Code Implementation

Problem - 4: Divide and Conquer in String Compression

Modern compression algorithms like LZ77 and Burrows–Wheeler Transform (BWT) rely heavily on substring repetition detection.

Task:

Design a **divide and conquer algorithm** that:

1. Divides a string into sub-blocks.
2. Finds the longest repeated substring efficiently across sub-blocks.
3. Combines results to produce a compressed representation.

Explain:

- How the merge step ensures correctness.
- The algorithm’s asymptotic time complexity.
- How this method compares with suffix-array-based compression.